In [3]:
```
# ========================================
# K-Means Clustering Assignment Solution
# Dataset: Mall Customer Segmentation (Kaggle)
# ========================================

# 1. Importing Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# For displaying plots inline in Colab
%matplotlib inline
```

In [4]:
```
# Load dataset (make sure the file is uploaded to your Colab environment)
df = pd.read_csv('/content/Mall_Customers.csv')  # adjust path if needed

# Preview data
print("Dataset Preview:")
display(df.head())

# Dataset info
print("\nDataset Info:")
print(df.info())

# Summary statistics
print("\nSummary Statistics:")
display(df.describe())
```

```
Dataset Preview:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None
```

Summary Statistics:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

In [5]:
```python
# Check for missing values
print("Missing Values:\n", df.isnull().sum())

# Drop non-numeric columns not needed for clustering (like CustomerID)
data = df.drop(columns=['CustomerID'])

# Encode categorical 'Gender' column (if exists)
data['Gender'] = data['Gender'].map({'Male': 0, 'Female': 1})

# Normalize numerical features
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data[['Age', 'Annual Income (k$)', 'Spendir

scaled_df = pd.DataFrame(scaled_data, columns=['Age', 'Annual Income (k$)', 'S

print("\nScaled Data Sample:")
display(scaled_df.head())

# We choose:
```

```
# Age

# Annual Income (k$)

# Spending Score (1-100)

# These features capture customer behavior, such as financial capacity and spe
```

```
Missing Values:
 CustomerID                0
Gender                     0
Age                        0
Annual Income (k$)         0
Spending Score (1-100)     0
dtype: int64
```

Scaled Data Sample:

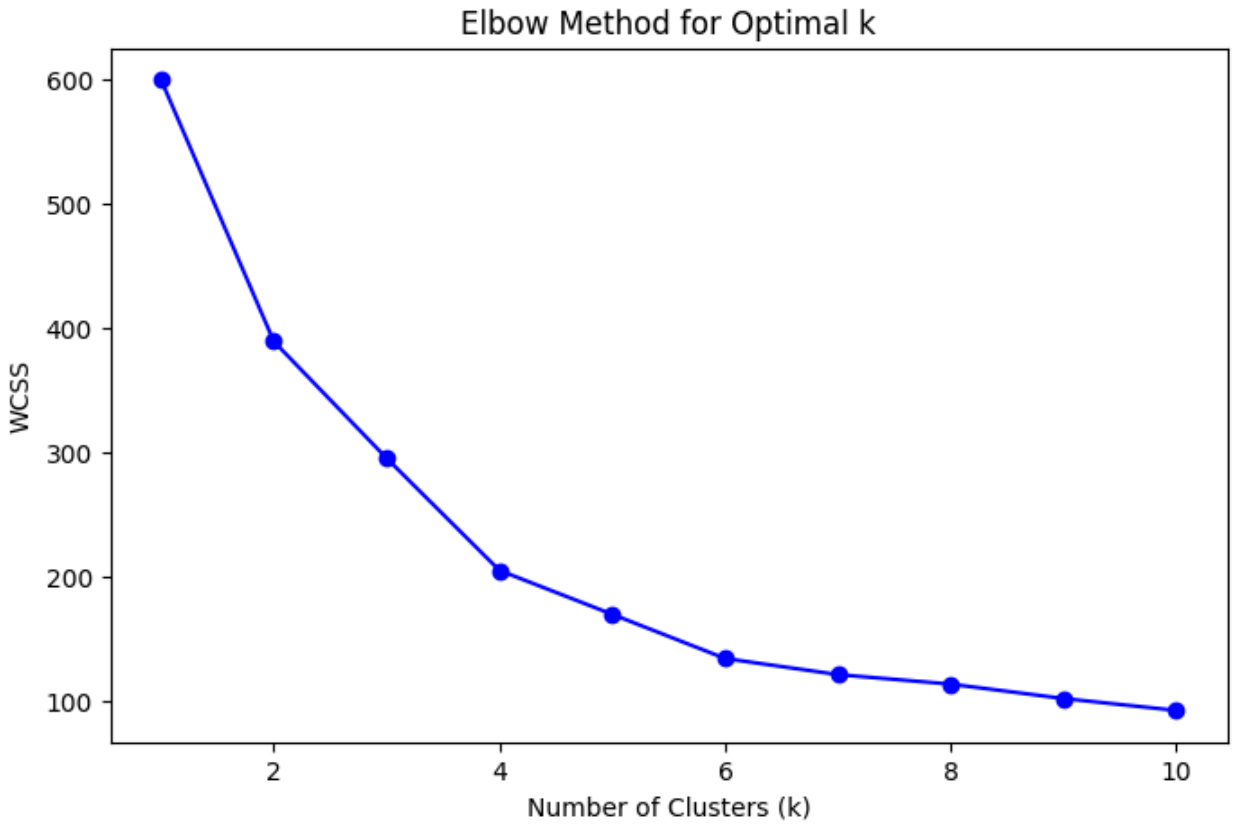
	Age	Annual Income (k$)	Spending Score (1-100)
0	-1.424569	-1.738999	-0.434801
1	-1.281035	-1.738999	1.195704
2	-1.352802	-1.700830	-1.715913
3	-1.137502	-1.700830	1.040418
4	-0.563369	-1.662660	-0.395980

In [6]:

```
# Elbow Method to Determine Optimal K
wcss = []  # Within-Cluster Sum of Squares
K = range(1, 11)

for k in K:
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(scaled_df)
    wcss.append(model.inertia_)

# Plot Elbow Method
plt.figure(figsize=(8,5))
plt.plot(K, wcss, 'bo-')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS')
plt.show()
```



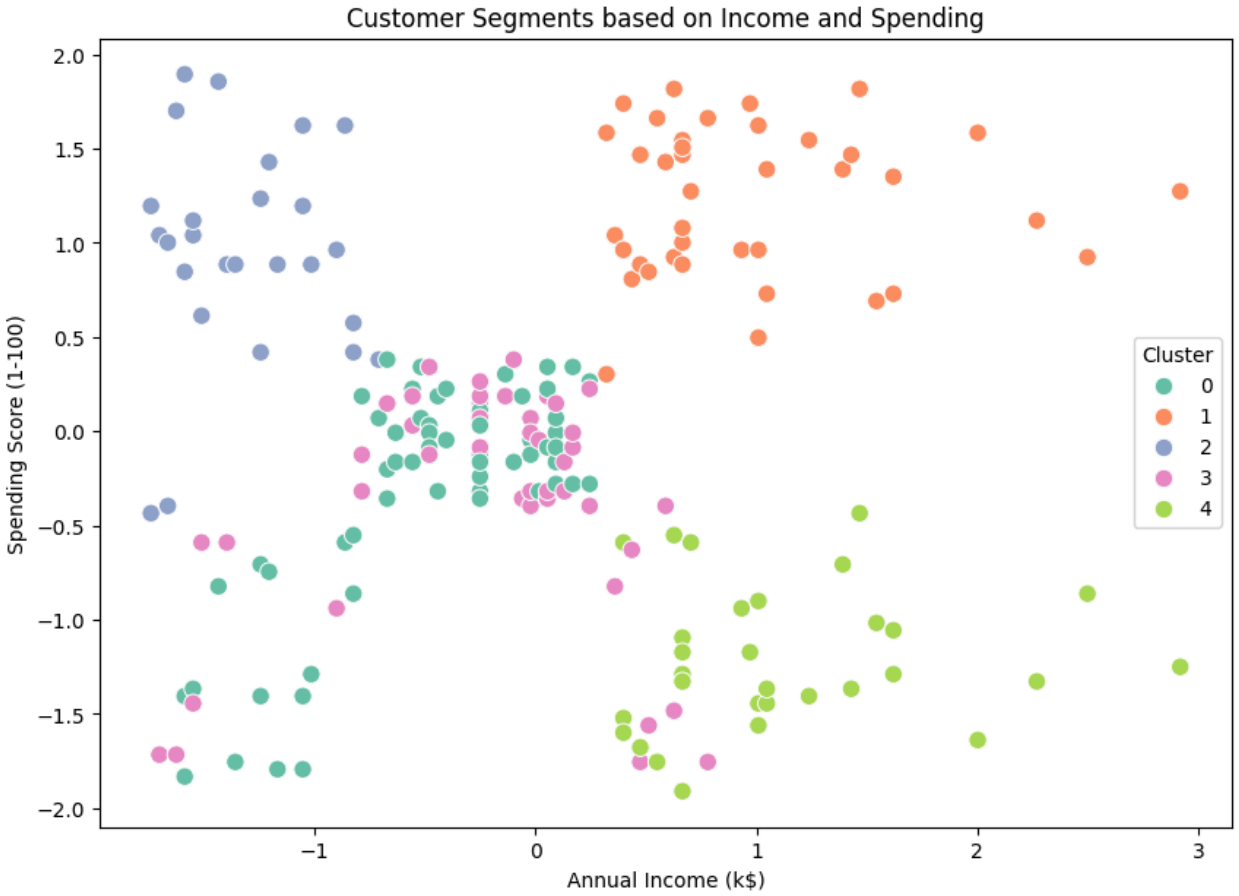
In [7]:
```
# Fit model with chosen k (say 5)
kmeans = KMeans(n_clusters=5, random_state=42)
kmeans.fit(scaled_df)

# Assign cluster labels
scaled_df['Cluster'] = kmeans.labels_

# Add cluster column to original data
data['Cluster'] = kmeans.labels_

# Visualize clusters
plt.figure(figsize=(10,7))
sns.scatterplot(data=scaled_df, x='Annual Income (k$)', y='Spending Score (1-1
                hue='Cluster', palette='Set2', s=80)
plt.title('Customer Segments based on Income and Spending')
plt.show()

# Silhouette score
score = silhouette_score(scaled_df.drop(columns=['Cluster']), scaled_df['Clust
print(f"Silhouette Score: {score:.3f}")
```



Silhouette Score: 0.408

In [9]:
```
# Cluster summaries
cluster_summary = data.groupby('Cluster')[['Age', 'Annual Income (k$)', 'Spend
display(cluster_summary)
```

Cluster	Age	Annual Income (k$)	Spending Score (1-100)
0	55.275862	47.620690	41.706897
1	32.875000	86.100000	81.525000
2	25.769231	26.115385	74.846154
3	26.733333	54.311111	40.911111
4	44.387097	89.774194	18.483871